

Chapter 10 Graphs

Chapter Summary

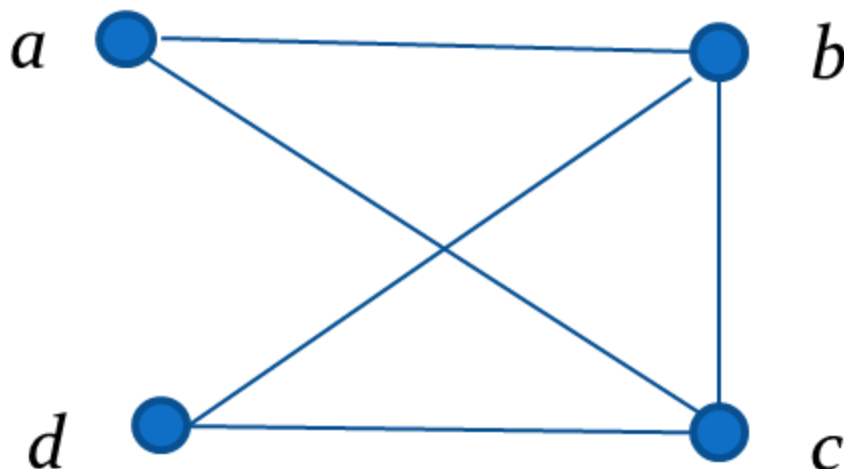
- Graph and Graph Models
- Graph Terminology and Special Types of Graphs
- Representing Graphs and Graph Isomorphism
- Connectivity
- Euler and Hamiltonian Graphs
- Shortest-Path Problem
- Planar Graphs
- Graph Coloring

Section 10.1 Graphs and Graph Models

Section Summary

- Introduction to Graphs
- Graph Taxonomy
- Graph Models

Graphs



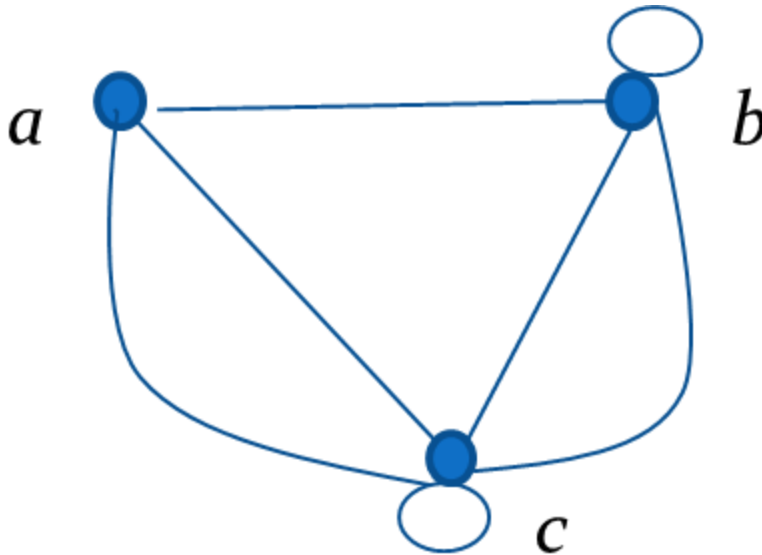
Definition:

$G = (V, E)$ where V is nonempty set of vertices and E is the set of edges. An edge connect its endpoints.

(Here we focus on graph with finite vertex set)

Some Terminology

- Simple Graph: Each edge connects two different vertices and no two edges connect the same pair of vertices.
- Multigraphs: May have multiple edges connecting two same vertices but no loops. If m different edges connect vertices u and v , we say $\{u, v\}$ is an edge with multiplicity m .
- Loop: An edge connects a vertex to itself.
- Pseudograph: May include loops or multiple edges.



Directed Graphs

Definition:

E is a set of directed edges. Each edge is associated with an ordered pair of vertices. (u, v) represents that start at u and end at v .

For undirected graphs, the edges are not directed and the point pair are not ordered.

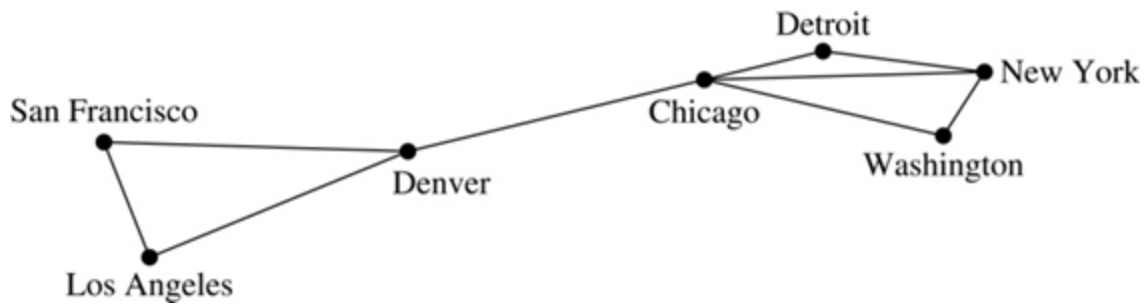
Some other Terminology

- Simple directed graph: no loops and no multiple edges.
- Directed multigraph: May have multiple directed edges but no loops.
- Mixed graph: May have directed and undirected edges, multiple edges and loops

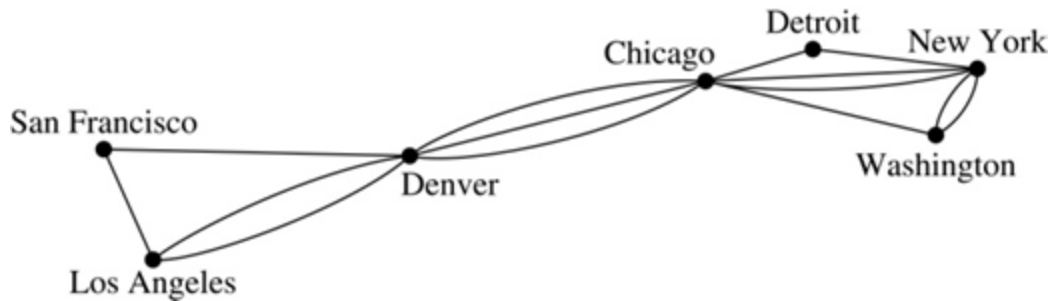
Graph Models: Computer Networks

The vertices represent data centers and the edges represent communication links.

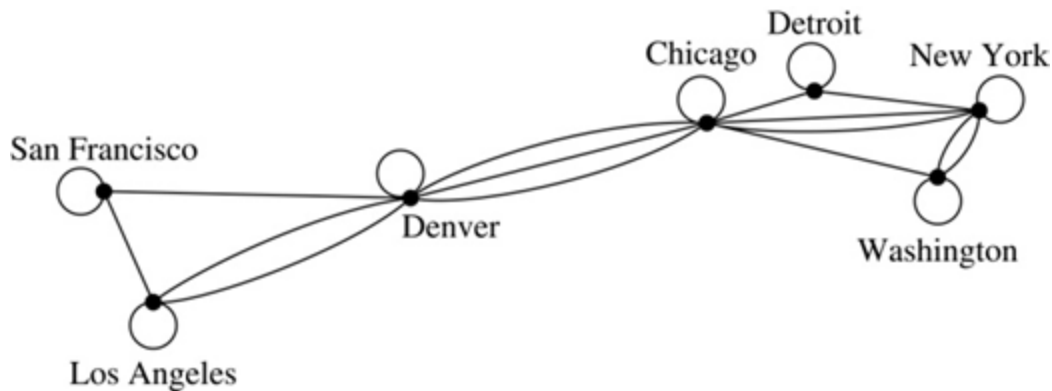
To model a computer network where we are only concerned whether two data centers are connected by a communications link, we use simple graph.



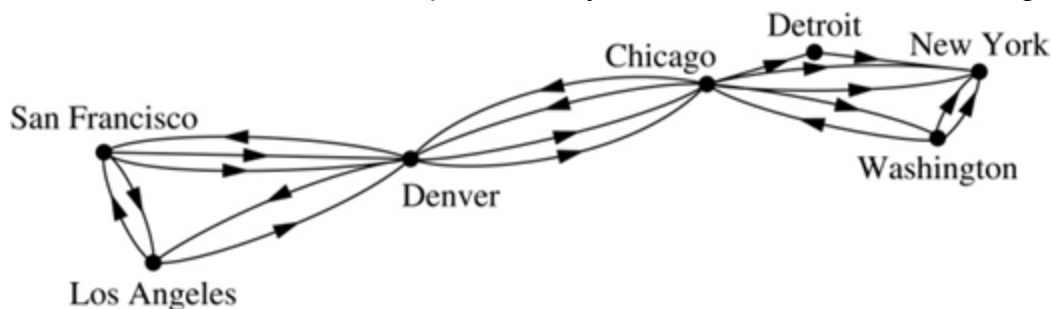
To model a computer network where we care about the number of links between data centers we use **multigraph**.



To model a computer network with diagnostic links at data centers, we use **pseudograph** as loops are needed.



To model a network with multiple one-way links, we use a directed multigraph.



Graph Terminology: Summary

- Undirected or directed or both
- Multiple edges or not
- loops or not

Other Applications of Graph

- Social networks
- Communication networks
- Information networks
- Software design
- Transportation networks
- Biological networks

Section 10.2 Graph Terminology and Special Types of Graphs

Section Summary

- Basic Terminology
- Some special types of graphs
- Bipartite graphs
- bipartite graphs and matchings
- Some applications of Special types of graphs
- New graphs from old

Basic Terminology

Definition 1:

Two vertices u, v in undirected graphs are called **adjacent or neighbors** in G if there is an edge e between u and v . e is called **incident with** the vertices u and v . And e is said to **connect** u and v .

Definition 2:

The set of all neighbors of v denoted by $N(v)$, called the **neighborhood** of v . if A is a subset of V , we denote $N(A)$ as the set of all vertices that are adjacent to at least one vertex in A . Thus $N(A) = \cup_{v \in A} N(v)$.

Definition 3:

The **degree** of a vertex in undirected graphs is the number of edges incident with it, denoted by $\deg(v)$. A loop contributes two to the degree.

Degrees of Vertices

Theorem 1(Handshaking Theorem):

If G is undirected graphs with m edges, then

$$2m = \sum_{v \in V} \deg(v)$$

Theorem 2:

An undirected graph has an even number of vertices of odd degree.

Directed Graphs

Definition:

The in-degree of a vertex v , denoted by $\deg^-(v)$ is the number of edges which terminate at v .

The out-degree of v , denoted by $\deg^+(v)$ is the number of edges which start at v . A loop contribute 1 in-degree and 1 out-degree.

Theorem 3:

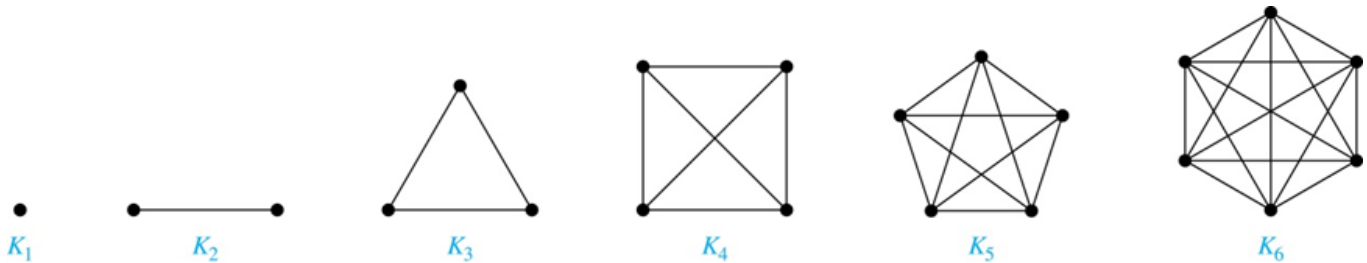
G is a directed graph, then

$$|E| = \sum_{v \in E} \deg^-(v) = \sum_{v \in V} \deg^+(v)$$

Special Types of Simple Graphs

Complete Graphs

Every pair of distinct vertices contains exactly one edge. Complete graphs with n vertices denoted by K_n .

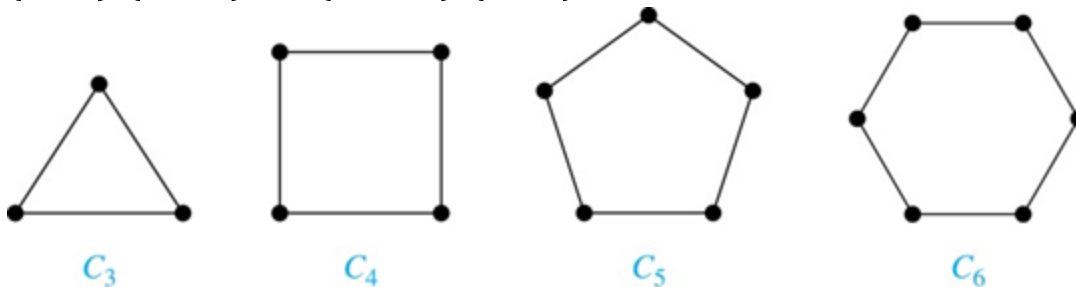


Cycles and Wheels

Cycle C_n :

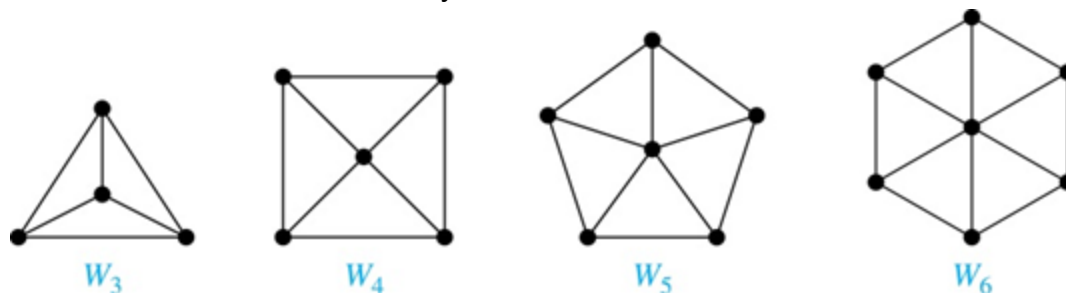
For $n \geq 3$ with n vertices $v_1, v_2, \dots, v_n$, consist the edges

$\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$.



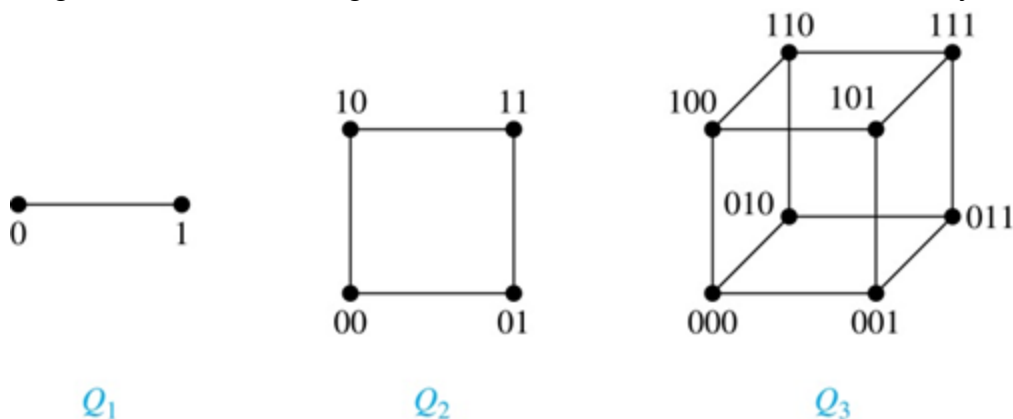
Wheel W_n :

Add an additional vertex to a cycle C_n and connect this new vertex to each of the n vertices.



n -Cubes

n -dimensional hypercube or n -cube, denoted by Q_n with 2^n vertices representing all bit string of length n . There is an edge between two vertices that differ in exactly one bit position.



Computer Network Architecture

- Star topology. All devices are connected to a central control device.
- Ring topology. Each device is connected using C_n .
- W_n -based topology. Combining features of star topology and ring topology.

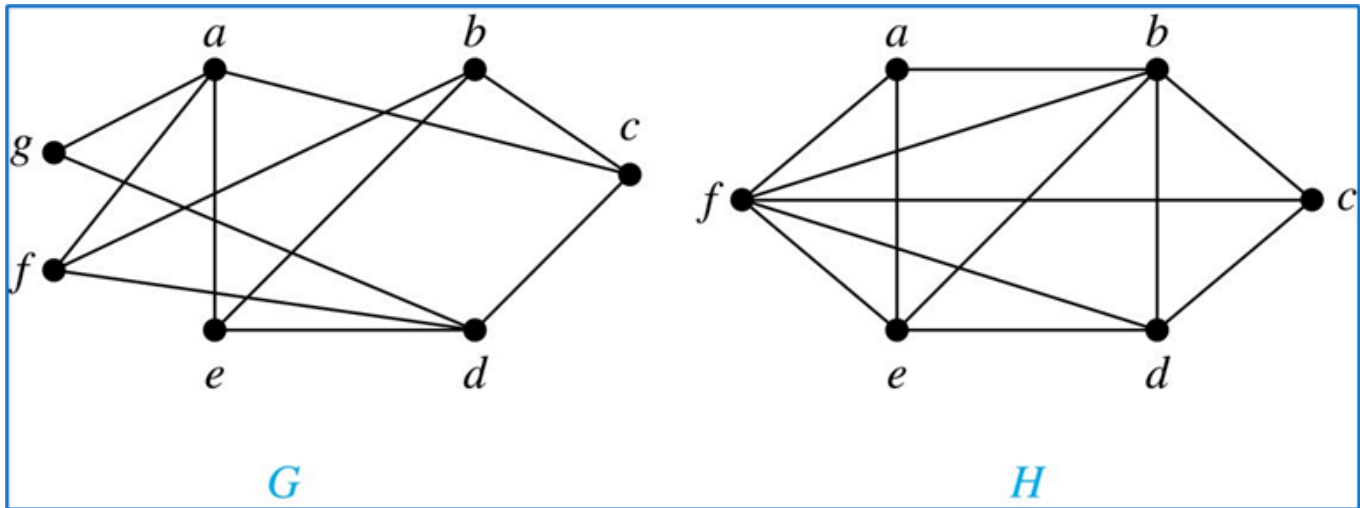
Bipartite Graphs

Definition:

A simple graph G with V be partitioned into two disjoint subset V_1, V_2 such that every edge connects a vertex in V_1 and a vertex in V_2 .

When showing the bipartite graphs, use two different colors to represent vertices. No two

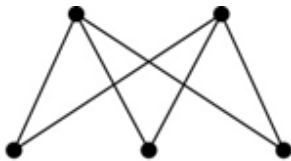
adjacent vertices are the same color.



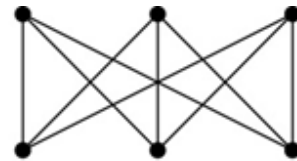
Complete Bipartite Graphs

Definition:

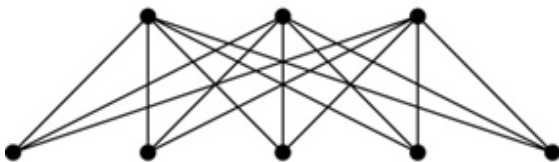
A graph that V_1 with size m and V_2 with size n such that there is an edge from every vertex in V_1 to every vertex in V_2 , denoted by $K_{m,n}$



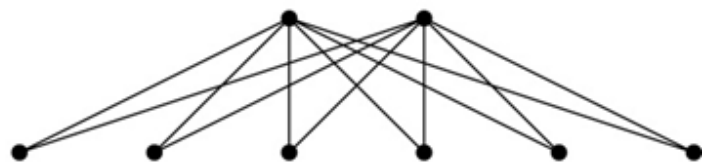
$K_{2,3}$



$K_{3,3}$



$K_{3,5}$



$K_{2,6}$

New Graphs from Old

Definition of subgraph:

A **subgraph** of $G = (V, E)$ is $H = (W, F)$ where $W \subset V$, $F \subset E$. It is **proper subgraph** if $H \neq G$.

Definition of induced subgraph:

The **subgraph induced** by a subset W of vertex set V is graph (W, F) where F contains an edge in E if and only if both endpoints are in W .

Definition of union of two simple graphs:

The union of $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edges set $E_1 \cup E_2$, denoted by $G_1 \cup G_2$.

Section 10.3 Representing Graphs and Graph Isomorphism

Section Summary

- Adjacency Lists
- Adjacency Matrices
- Incidence Matrices
- Isomorphism of graphs

Adjacency List

Definition:

It can be used to represent a graph with **no multiple edges** by specifying the vertices that are adjacent to each vertex of the graph.

TABLE 1 An Adjacency List for a Simple Graph.	
<i>Vertex</i>	<i>Adjacent Vertices</i>
<i>a</i>	<i>b, c, e</i>
<i>b</i>	<i>a</i>
<i>c</i>	<i>a, d, e</i>
<i>d</i>	<i>c, e</i>
<i>e</i>	<i>a, c, d</i>

Adjacency Matrices

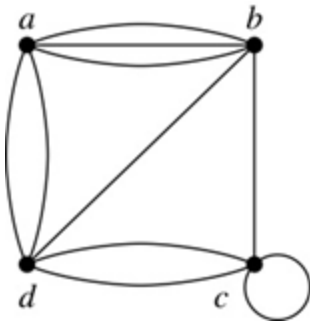
Definition:

Suppose that $G = (V, E)$, $|V| = n$, the adjacency matrix A_G of G with respect to the listing of vertices is $n \times n$ zero-one matrix.

$$a_{ij} = \begin{cases} 1, & \text{if } \{v_i, v_j\} \text{ is an edge of } G \\ 0, & \text{otherwise} \end{cases}$$

When the graph is sparse, use adjacency list; when the graph is dense, use adjacency matrices.

Adjacency Matrix can also be used to represent graphs with **loops and multiple edges**. A loop is represent by 1 at (i, i) . (i, j) represent the number of edges.



$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

When representing **directed graph**, (i, j) represent the edge from v_i to v_j .

Incidence Matrices

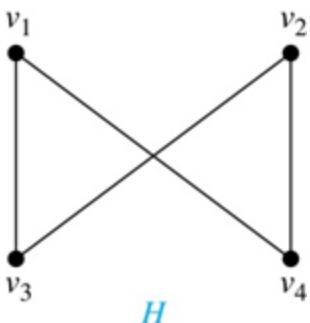
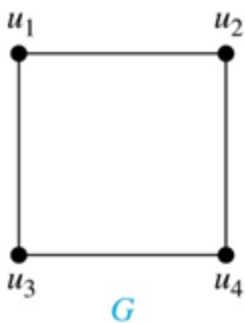
$G = (V, E)$ with n vertices and m edges. The incidence matrix with respect to the ordering of V and E is the $n \times m$ matrix $M = [m_{ij}]$ where

$$m_{ij} = \begin{cases} 1, & \text{when edge } e_j \text{ is incident with } v_i \\ 0, & \text{otherwise} \end{cases}$$

Isomorphism of Graphs

Definition:

For simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, they are **isomorphic** if there is a bijective function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 . f is called **isomorphism**.



$$f(u_1) = v_1, f(u_2) = v_4, f(u_3) = v_3, f(u_4) = v_2.$$

It is difficult to determine isomorphism using brute force ($n!$ possible situations). Best algorithm have exponential worst case. It is one of a few NP problems not known to be either tractable or NP-complete.

Sometimes it is not hard to show two graphs are not isomorphic. We can find **graph invariant** that is property preserved by isomorphism like the number of vertices, edges and degree sequence.

Section 10.4 Connectivity

Section Summary

- Paths
- Connectedness in Undirected graphs
- Vertex Connectivity and Edge connectivity
- Connectedness in Directed graphs
- Paths and Isomorphism
- Counting Paths between Vertices

Paths

Definition:

A **path of length n from u to v** in G is a sequence of n edges $e_1, \dots, e_n$ of G such that e_i has the endpoints x_{i-1} and x_i . ($x_0 = u, x_1, \dots, x_{n-1}, x_n = v$)

If the graph is simple, we denote the path by vertex sequence $x_0, x_1, \dots, x_n$.

If $u = v$, the path is a **circuit**. It is said to **pass through** $x_1, \dots, x_{n-1}$ and **traverse** the edges $e_1, \dots, e_n$. A path or circuit is **simple** if it does not contain the same edge more than once.

Degrees of Separation

Paths in Acquaintanceship graphs.

Erdos Numbers and Bacon Numbers

Two special length of shortest path.

Connectedness in Undirected Graphs

Definition:

An undirected graph is called **connected** if **there is a path between every pair of vertices**.

Connected Components

Definition:

A **connected component** of G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G . A graph is not connected(or disconnected) if it has two or more connected components.

Connectedness in Directed Graphs

Definition:

A directed graph is **strongly connected** for every pair of u and v if there is a path from u to v and a path from v .

A directed graph is **weakly connected** if there is a path between every two vertices in the underlying undirected graph.

Definition:

The maximal strongly connected subgraphs are called the **strongly connected components** or **strong components** of G .

Counting Paths between Vertices

Theorem:

Let G be a graph with adjacency matrix A with n vertices(directed or undirected, multiple edges and loops allowed). The number of **different paths** of length r from v_i to v_j equals to (i, j) entry of A^r .

Proved by mathematical induction.

Section 10.5 Euler and Hamiltonian Graphs

Section Summary

- Euler Paths and Circuits
- Hamilton Paths and Circuits
- Applications of Hamilton Circuits

Euler Paths and Circuits

Definition

An **Euler circuit** in a graph G is a simple circuit containing **every edge** of G . An **Euler path** in G is a simple path containing **every edge** of G .

Conditions for Euler Circuits and Paths

Euler circuits:

The **degree** of every vertex **must be even**.

Euler paths:

Has exactly two vertices of odd degree as initial vertex and final vertex.

Algorithm for Constructing an Euler Circuits

For an connected multigraph with all vertices of even degree.

- Choose an **arbitrary vertex as initial vertex**.
- Choose vertex with edges successively and **add to form a path that returns to initial vertex**.
- If there exist other edges in G , **construct a subcircuit** beginning at a vertex **in the rest graph that is also an endpoint in circuit**.
- **Insert** the subcircuit at appropriate position in circuit and **remove** corresponding vertices and edges in the rest graph.
- Repeat until all vertices are included.

Applications of Euler Paths and Circuits

- Finding a path or circuit that traverses each
- Layout of circuits
- Network multicasting
- Molecular biology

Hamilton Paths and Circuits

Definition:

A simple path in G that passes through every vertex exactly once is called a **Hamilton path**. And a simple circuit in G that passes through every vertex exactly once is called a **Hamilton circuit**.

Necessary Conditions for Hamilton Circuits

No simple **necessary and sufficient conditions** for the existence of Hamilton circuit.

Necessary conditions:

- Dirac's Theorem:
 G is a simple path with $n \geq 3$ vertices such that degree of every vertex in G is $\geq n/2$, then G has a Hamilton circuit.
- Ore's Theorem:
 $\deg(u) + \deg(v) \geq n$ for each pair of **nonadjacent vertices**, then G has a Hamilton circuit.

Applications of Hamilton Paths and Circuits

- Traveling salesperson problem(TSP)
- Gray codes